

01 포지셔닝

CLAIM

AI 기능을 실제 Android 사용자 흐름에서 검증 가능한 제품으로 만드는 개발자.

EVIDENCE

Kotlin/Compose/CameraX/ML Kit/MediaProjection/Overlay/Flavor/QA evidence를 하나의 케이스로 정리.

DETAIL

저는 모델 API 호출보다 권한 UX, 입력 품질, 결과 렌더링, 실패 로그까지 함께 설계하는 개발자로 포지셔닝합니다.
이 프로젝트는 앱개발과 AI서비스 개발 직무를 동시에 설명할 수 있는 대표 사례입니다.

02 문제 정의

CLAIM

외국어 화면을 이해하는 과정의 전환 비용을 줄이는 모바일 AI 도구.

EVIDENCE

홈 화면의 세 핵심 행동: 촬영해서 번역, 사진 불러오기, 화면 위 번역.

DETAIL

일반 번역앱은 사용자가 촬영, 복사, 붙여넣기, 앱 전환을 반복해야 합니다. LensOverlay는 카메라/갤러리/현재 화면이라는 실제 입력 흐름을 먼저 정의하고, AI/OCR은 그 흐름을 지원하는 내부 파이프라인으로 배치했습니다.

03 사용자 흐름

CLAIM

첫 화면에서 다음 행동이 바로 보여야 실제 제품처럼 보입니다.

EVIDENCE

UIAutomator dump에서 LensOverlay Translate, 촬영해서 번역, 사진 불러오기, 화면 위 번역 텍스트 확인.

DETAIL

기능명보다 행동명을 앞에 배치했습니다. 사용자는 촬영, 이미지 선택, 화면 위 번역 중 하나를 선택하고, 앱은 OCR /번역/결과 표시를 이어서 처리합니다. 이 구성이 포트폴리오 첫 장에서도 바로 이해되도록 문서 구조를 맞췄습니다.

04 실행 화면

CLAIM

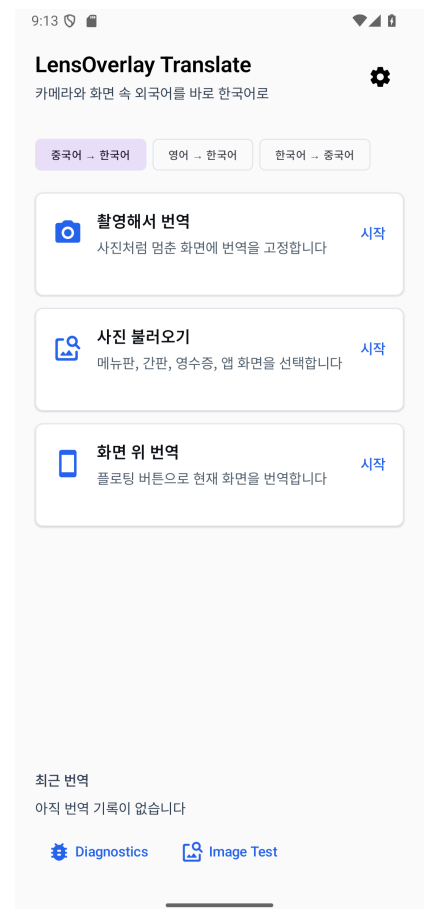
실행 가능한 APK와 화면 증거를 함께 제출합니다.

EVIDENCE

Medium_Phone_API_35, emulator-5554, com.lensoverlay.translate.china, versionName 0.1.0-china 실행 스크린샷.

DETAIL

오른쪽 화면은 검증 대상 APK의 홈 화면입니다. debug build라 Diagnostics /Image Test가 보이지만, 이 진입점은 QA evidence로 분리하고 일반 사용자 화면에서는 숨겨야 합니다.



05 아키텍처

CLAIM

입력, OCR, 번역, 렌더링, QA를 분리해 설명 가능한 구조로 만들었습니다.

EVIDENCE

MainActivity, CameraScreen, ScreenCaptureForegroundService, OverlayBubbleService, MlKitOcrEngine, TranslationRouter 확인.

DETAIL

Compose UI -> CameraX/MediaProjection -> ML Kit OCR -> OCR candidate filtering -> TranslationRouter -> ResultSheet/overlay rendering -> QA diagnostics 흐름입니다. 현업 면접에서 각 단계별 tradeoff를 설명할 수 있게 구성했습니다.

06 Android 권한

CLAIM

화면 캡처와 오버레이는 권한 UX가 제품 품질의 핵심입니다.

EVIDENCE

SYSTEM_ALERT_WINDOW, FOREGROUND_SERVICE_MEDIA_PROJECTION, POST_NOTIFICATIONS, CAMERA 권한 확인.

DETAIL

권한을 숨기거나 우회하지 않고 명시적 동의, foreground notification, 즉시 종료 흐름을 전제로 잡았습니다. 포트폴리오에서는 이것을 보안/개인정보 감각의 증거로 제시합니다.

07 번역 파이프라인

CLAIM

AI/OCR은 호출보다 후보 품질과 실패 처리까지 포함해야 합니다.

EVIDENCE

ML Kit OCR recognizer: latin/chinese/japanese/korean. TranslationRouter와 RemoteTranslationClient 구조 확인.

DETAIL

OCR 후보 중 실제 번역할 텍스트를 고르고, 이미 한국어인 텍스트와 노이즈를 줄이는 전략을 씁니다. market flavor에 따라 provider 경로를 분리해 유지보수와 리스크를 낮춥니다.

08 QA 증거

CLAIM

주장보다 증거가 먼저 보이는 포트폴리오로 바꿨습니다.

EVIDENCE

SHA-256 031DD650F33A0E94CFE279F4F2CECA7A6E8B8519C67C7ED113C8455588B5AF3F; install Success; MainActivity focus; targetSdk 35; logcat app FATAL/ANR 미검출.

DETAIL

Gradle test/build, APK metadata, emulator install/run, screenshot, UIAutomator dump, logcat 필터를 evidence folder에 모았습니다. 면접관이 질문하면 바로 근거 파일을 열어 설명할 수 있습니다.

09 리스크

CLAIM

대기업 면접에서는 미완료 리스크를 숨기지 않는 편이 더 강합니다.

EVIDENCE

Physical device QA, China network/provider latency, release signing, privacy notice, server auth/rate limit 미완료.

DETAIL

상용 배포 완료라고 말하지 않습니다. 현재는 debug demo와 emulator evidence가 있는 단계입니다. 남은 리스크를 30일/90일 실행 계획으로 바꿔 제품화 감각을 보여줍니다.

10 내 기여

CLAIM

제품 흐름, Android 권한, AI 파이프라인, QA 증거를 한 번에 묶었습니다.

EVIDENCE

복사본 source 기준 main java files 52개, test files 8개, QA_REPORT의 반복 build/test evidence.

DETAIL

제가 강조할 기여는 AI를 붙임이 아니라 사용자가 누르는 흐름부터 검증 결과까지 이어지는 제품 단위 구현입니다.
이것이 앱 개발과 AI 서비스 개발 직무 모두에 연결됩니다.

11 직무 적합성

CLAIM

Android/AI 서비스 조직에서 바로 질문받을 수 있는 실전형 포트폴리오입니다.

EVIDENCE

권한 처리, OCR 품질, provider routing, APK 검증, emulator QA, 개인정보 리스크까지 질문 포인트 제공.

DETAIL

대기업 현업 면접관은 구현 결과보다 의사결정과 tradeoff를 봅니다. 이 패키지는 왜 이렇게 나눴는가, 무엇을 검증했는가, 무엇이 아직 위험한가를 한 문서 안에서 답하게 설계했습니다.

12 다음 90일

CLAIM

제출용 PASS에서 제품 파일럿 PASS로 가는 계획이 명확합니다.

EVIDENCE

30일: actual devices, QA-signed release, p50/p95 latency. 90일: user tests, crash-free/latency dashboard, privacy/server cost guard.

DETAIL

다음 단계는 실제 기기 2종 이상, 중국망/provider 실측, release signing, 개인정보 고지, 서버 proxy 인증/요금 제한입니다. 이 계획을 면접에서 말하면 성숙도가 올라갑니다.